Hackathon API Guide

This guide is for teams building dashboards, integrations, and robot workflow experiences against the ACU API Gateway from their own laptops or applications. It covers the current robot/PWA workflow APIs and the supporting HTTP APIs used by the operator PWA (`ACU.FE`) and robot PWA (`Robot.Pwa.App`). Legacy workflow APIs and WebSocket integration are out of scope.

The routes below were checked against the gateway source on 2026-10-22. The organizer's running Swagger documentation is the final reference for the version deployed at the event.

1. Connect from your laptop

Get these details from the organizers

Detail	Example / purpose
Gateway base URL	`http://10.27.238.122/docs/index.html` (replace with the actual address)
Network access	Event Wi-Fi/LAN or organizer-provided VPN
Assigned device ID	A robot or simulator your team is allowed to use
Write permissions	Which approval, assignment, settings, and deletion operations your team may perform
Deployment access requirements	Any authentication or HTTPS enforced by an organizer's reverse proxy

Use the gateway host's address, not `localhost`. On your laptop, `localhost` means your laptop, not the organizer's machine. `0.0.0.0` is a server listen address, not a client destination. The robot's IP is also not the gateway URL.

The default direct gateway port is **3000**. With the root Docker Compose setup, the operator UI is on port **3100**; that is not the direct API port.

First connectivity check

Run in PowerShell, replacing the example host:

```
$baseUrl = "http://192.168.1.100:3000"
Test-NetConnection -ComputerName "192.168.1.100" -Port 3000
Invoke-RestMethod -Uri "$baseUrl/api/health"
```

Expected health response (`timestamp` changes):

```
{
  "status": "ok",
  "timestamp": "2026-10-22T12:30:00.0000000Z"
}
```

Open these URLs in your browser:

URL	Purpose
`<baseUrl>/docs`	Swagger UI: browse schemas and try HTTP requests
`<baseUrl>/swagger/v1/swagger.json`	OpenAPI document; import this URL into Postman
`<baseUrl>/api/health`	Lightweight reachability check

In Postman, create a `baseUrl` environment variable and use URLs such as `{{baseUrl}}/api/acu/devices`. Choose **Body > raw > JSON** for JSON payloads, and **Body > form-data** for media uploads.

Organizer network checklist

- Run the gateway on an interface reachable from the event network. The checked-in development Kestrel endpoint listens on `http://0.0.0.0:3000`; verify the deployed settings and startup logs.
- Allow inbound TCP traffic to the gateway port on the host firewall, scoped to the event network. Publish the port when running inside Docker.
- Ensure Wi-Fi client isolation, VPN routing, and host firewall rules permit participant-to-gateway traffic. Being on the same SSID alone is not sufficient.
- The current gateway allows any CORS origin, method, and header. A deployed proxy can impose additional rules.
- An HTTPS frontend cannot normally fetch an HTTP gateway because of browser mixed-content restrictions. Provide HTTPS for the gateway or a same-origin HTTPS proxy; do not disable browser security.
- Verify access from a second machine before distributing the base URL.

The current gateway code does **not** implement API-key/JWT/login authentication. Device approval and MAC checks are application-state checks, not user access control. Keep the event gateway on a trusted network/VPN; do not expose it unprotected to the public internet. Follow any extra access controls configured by the organizers.

2. Request conventions and shared-system rules

- All route paths below are relative to `baseUrl`; include `/api` exactly once.
- Use `Content-Type: application/json` for JSON bodies. Bodyless operations need no JSON body. Uploads use `multipart/form-data`, with the client supplying the boundary.
- Most JSON uses camelCase. **Robot workflow documents use explicitly named, case-sensitive output keys such as `WorkflowID`, `RobotId`, `Steps`, and `Workflows`.** Copy the examples rather than converting them to camelCase.
- URL-encode device and workflow IDs when constructing URLs. Use team-prefixed IDs such as `team-blue-welcome` to avoid collisions.

- `PUT` workflow/settings requests replace the resource; send the complete document, not just changed fields. Telemetry updates are the exception: supplied telemetry keys are merged.
- `204 No Content` is a successful response with **no JSON body**.
- Optional null properties are generally omitted from responses; do not assume every device has telemetry, a location, or an assignment.
- For dashboards, polling approximately every 3 seconds is a reasonable starting point, with backoff on failures. Do not busy-loop against a shared gateway.

Shared robots: only control your assigned device. Approval/rejection, commands, workflow publishing, destination replacement, and deletion change shared state. Settings affect **every robot**. Coordinate physical movement with the organizers; a successful HTTP write does not prove a robot executed it.

3. Choose the API for your problem statement

Goal	Start with
Build a fleet dashboard	`GET /api/acu/devices`
Show battery, location, or event history	Device list fields `statusData`, `lastSeenLocation`, `recentEvents`
View available robot destinations	`GET /api/bot/destinations/{deviceId}`
Build a robot/PWA screen sequence	Robot workflows in section 5
Upload workflow video/audio/images	Media APIs in section 6
Integrate a simulator or robot client	Registration and telemetry in section 7
Change robot speech language	Settings in section 8; organizer approval recommended

4. Devices, approval, telemetry reads, and commands

Method	Route	Body	Success / behavior
GET	`/api/health`	None	`200`: `{ "status": "ok", "timestamp": "..." }`
GET	`/api/acu/devices`	None	`200`: array of all devices, including pending/rejected ones
POST	`/api/acu/accept/{deviceId}`	None	`200`: `{ "message": "Device approved", "deviceId": "..." }`; unknown device: `404`
POST	`/api/acu/reject/{deviceId}`	None	`200`: `{ "message": "Device rejected", "deviceId": "..." }`; unknown device: `404`
GET	`/api/acu/device-status/{deviceId}`	None	`200`: raw telemetry dictionary when present; unapproved: `403`; unknown device: `404`

Method	Route	Body	Success / behavior
POST	`/api/acu/instructions/{deviceId}`	`{ "command": "..." }`	`200`: `{ "message": "Instructions saved" }`; invalid body: `400`; unapproved: `403`; unknown device: `404`; offline: `503`

Device list response

An illustrative device entry (telemetry keys depend on the robot):

```
[
  {
    "deviceId": "robot-001",
    "name": "Demo Robot",
    "ip": "192.168.1.42",
    "mac": "AA:BB:CC:DD:EE:FF",
    "firmware": "v1.2.3",
    "registeredAt": "2026-10-22T12:00:00Z",
    "status": "approved",
    "lastSeen": "2026-10-22T12:30:00Z",
    "lastSeenLocation": "Reception",
    "online": true,
    "statusData": { "battery": 85, "event": "patient_arrived" },
    "instructions": "",
    "recentEvents": [
      { "event": "patient_arrived", "receivedAt": "2026-10-22T12:30:00Z" }
    ],
    "destinations": [
      { "name": "Reception", "type": "usher", "x": 1.0, "y": 2.0 }
    ]
  }
]
```

- `status`: registration approval state (`pending`, `approved`, `rejected`).
- `online`: recent device activity, **not** proof of readiness for movement. The server uses a 20-second freshness threshold and checks about every 10 seconds.
- `statusData`: extensible telemetry, not a fixed battery/location schema. A device without telemetry can yield an empty response (`204`) from the raw status-read endpoint; handle that as well as `200`.
- `recentEvents`: up to 50 timestamped events. Consecutive duplicate event names are not appended. Workflow assignment/start operations can clear history.
- For robot/PWA assignments, query `/api/acu/robot-workflows/assignments`. Do not use the device's optional `assignedWorkflowId` field for this workflow format; it belongs to a separate, out-of-scope workflow system.

The operator PWA separates approved devices from pending/rejected requests using this single list. There is no separate request-list or instruction-log API.

Commands are non-empty strings interpreted by the receiving robot implementation; there is no gateway-defined command catalogue. Agree supported commands with the organizers. This endpoint stores the latest command; it is not a command queue, execution receipt, or guaranteed immediate delivery.

5. Robot/PWA workflows

Use this family for the current robot/PWA workflow format.

Workflow management APIs

Method	Route	Body	Success / behavior
POST	`/api/workflows/robot`	Robot workflow object below	`201`: created workflow; `Location` points to its read URL
GET	`/api/acu/robot-workflows`	None	`200`: array of saved robot workflow objects
GET	`/api/acu/robot-workflows/{id}`	None	`200`: one workflow; missing: `404`
PUT	`/api/acu/robot-workflows/{id}`	Complete robot workflow object	`200`: updated workflow; body `WorkflowID` must equal URL ID
DELETE	`/api/acu/robot-workflows/{id}`	None	`204`; also removes its robot/PWA assignments; missing: `404`
GET	`/api/acu/robot-workflows/assignments`	None	`200`: `{ "Workflows": [...] }`, containing unfinished assigned snapshots
POST	`/api/acu/robot-workflows/{id}/assignments/{deviceId}`	None	`204`: publish a snapshot to the device, replacing its previous robot/PWA assignment
DELETE	`/api/acu/robot-workflows/assignments/{deviceId}`	None	`204`: clear only the robot/PWA assignment; device must exist and be approved

Create/update/publish operations validate the robot and destinations: `400` for invalid workflow data, `403` for an unapproved robot, and `404` for a missing robot/workflow. Creating a duplicate `WorkflowID` returns `409`.

Minimal screen-only workflow

Replace `RobotId` with your organizer-assigned, **approved** device. POST this single object, **not** a `{ "Workflows": [...] }` wrapper:

```
{
  "WorkflowID": "team-blue-welcome",
  "WorkflowName": "Team Blue Welcome",
  "Description": "A screen-only hackathon demonstration",
  "RobotId": "robot-001",
  "Steps": [
    {
      "StepId": "Step-1",
      "ScreenId": "Confirmation",
      "Title": "Welcome to our demo",
      "TTS": "Welcome to our demo",
      "Buttons": {
        "Yes": { "Label": "Continue", "Action": "Next" },
        "No": { "Label": "Back", "Action": "Previous" }
      }
    }
  ]
}
```

Field	Meaning / rules
`WorkflowID`, `WorkflowName`, `RobotId`	Required non-empty strings; choose a unique team workflow ID
`Description`	Optional descriptive text
`Steps`	Required non-empty ordered array
`StepId`	Required; unique within the workflow
`ScreenId`	Required; identifies a screen template supported by the robot app, such as `Confirmation` or `FollowMe`
`Title`, `TTS`	Screen title and optional text-to-speech text
`StartDestinationName`, `NextDestinationName`	Optional names; every non-empty value must exactly match a destination stored for that robot, including case
`Video`, `Audio`, `Image`	Optional arrays of media objects, described in section 6
`Buttons`	Optional pair; if supplied, both `Yes` and `No` need non-empty `Label` values
`Action`	`Next` or `Previous`; defaults to `Next` if omitted

The gateway checks that `ScreenId` is non-empty, not that the robot implements that template. Confirm supported screen IDs with the organizers. For a `FollowMe` step, provide `NextDestinationName`; the operator editor treats it as a movement-only step, without title/media/buttons. Do not invent map names.

Publish and execution lifecycle

1. Read `/api/acu/devices` and identify your approved robot.
2. Read its destinations if the workflow needs movement.
3. Create the workflow with `POST /api/workflows/robot`.
4. Publish it with `POST /api/acu/robot-workflows/{id}/assignments/{deviceId}`.
5. The robot client reads the assigned document and starts it using the APIs below.
6. Monitor telemetry/events through `/api/acu/devices`.

Method	Route	Body	Success / behavior
GET	`/api/bot/robot-workflows/{deviceId}/workflow`	None	`200`: `{ "Workflows": [<assigned workflow>] }`, or `{ "Workflows": [] }` if none
POST	`/api/bot/robot-workflows/{deviceId}/workflow/start`	None	`200`: workflow document; clears event history; no assignment: `404`
POST	`/api/bot/robot-workflows/{deviceId}/workflows/{workflowId}/complete`	None	`204`: marks the current assignment completed; wrong/no current workflow: `409`

These three endpoints require a registered, approved robot (`404`/`403` otherwise). They are for robot clients; dashboard teams should not start or complete another client's workflow.

Important lifecycle details:

- Workflow definitions are saved to disk, but robot/PWA assignments are **in memory and lost on gateway restart**. Publish again after a restart.
- Assignment uses a snapshot. Editing a saved workflow does not change an existing assigned snapshot; publish again to deliver the update.
- Completion removes a workflow from the unfinished assignments listing, but the robot's GET still returns its last completed snapshot.
- Starting a completed assignment refreshes it from the saved definition.

6. Workflow media

Method	Route	Body	Success / behavior
POST	`/api/acu/robot-workflows/media`	Multipart fields `kind` and `file`	`201`: `{ "path": "/api/bot/robot-workflows/media/<generated-name>" }`
GET	`/api/bot/robot-workflows/media/{name}`	None	`200`: binary media, or `206` for a supported range request; missing/invalid name: `404`

`kind` must be exactly `Video`, `Audio`, or `Image`, and must match the extension:

Kind	Supported file extensions
Video	`.mp4`, `.webm`
Audio	`.mp3`, `.wav`, `.m4a`, `.ogg`
Image	`.png`, `.jpg`, `.jpeg`, `.webp`

Files must be non-empty and no larger than **250 MiB (262,144,000 bytes)**. Invalid kind/extension/size returns `400`; oversized HTTP requests can also be rejected by the server/proxy (for example `413`). Device playback still depends on its codec support.

Example upload from PowerShell using `curl.exe`:

```
curl.exe --fail-with-body -X POST "$baseUrl/api/acu/robot-workflows/media" `
  -F "kind=Image" -F "file=@C:\Hackathon\welcome.png"
```

Copy the returned path into the matching media array in a workflow step:

```
{
  "StepId": "Step-2",
  "ScreenId": "Confirmation",
  "Title": "About our project",
  "Image": [
    {
      "ImagePath": "/api/bot/robot-
workflows/media/0123456789abcdef0123456789abcdef.png",
      "TTS": "Here is our project"
    }
  ]
}
```

The filename above is illustrative; use the actual upload response. `Video` items require `VideoPath`, `Audio` items require `AudioPath`, and `Image` items require `ImagePath`. Each item may also have `TTS` and `Buttons` in the same format as a step. Resolve returned relative paths against the gateway base URL, not your frontend's host. There is no media-list or media-delete HTTP API.

7. Robot/simulator registration, telemetry, and destinations

Dashboard-only teams usually need only the device reads in section 4 and the destination GET below. Use these writes only for your team's simulator or with organizer permission; do not spoof telemetry for a real shared robot.

Method	Route	Body	Success / behavior
POST	`/api/bot/register`	Registration object below	`200`: pending registration; duplicate device ID/MAC: `409`; invalid body: `400`
GET	`/api/bot/registration-status/{deviceId}`	None	`200`: `{ "status": "pending", "deviceId": "..." }`; status can also be `approved`, `rejected`, or `unregistered`
DELETE	`/api/bot/unregister/{deviceId}`	`{ "mac": "AA:BB:CC:DD:EE:FF" }`	`200`: device removed; wrong MAC: `403`; unknown device: `404`
POST	`/api/bot/device-status/{deviceId}`	Telemetry object below	`200`: status acknowledgment; invalid fields: `400`; wrong supplied MAC/unapproved: `403`; unknown device: `404`
GET	`/api/bot/instructions/{deviceId}`	None; optional query `?mac=...`	`200`: `{ "command": "..." }` with optional additional assignment metadata; wrong supplied MAC/unapproved: `403`; unknown device: `404`
GET	`/api/bot/destinations/{deviceId}`	None	`200`: destination array; returns `[]` even for an unknown device
POST	`/api/bot/destinations/{deviceId}`	Destination array below	`200`: storage confirmation; invalid entry: `400`; unknown device: `404`

Registration body

`deviceId` and `mac` are required. Use unique organizer-approved identities. Registration does not approve the device or immediately make it online.

```
{
  "deviceId": "team-blue-sim",
  "name": "Team Blue Simulator",
  "ip": "192.168.1.50",
  "mac": "02:00:00:00:01:01",
  "firmware": "hackathon-1"
}
```

Wait for approval (or have an organizer call `/api/acu/accept/{deviceId}`). Use the registration-status GET while waiting; telemetry POST returns `403` until approved. Re-registering an existing ID returns `409`, even for the same MAC; do not unregister a shared device simply to bypass this.

Telemetry body and acknowledgment

```
{
  "mac": "02:00:00:00:01:01",
  "battery": 85,
  "lastSeenLocation": "Reception",
  "event": "demo_started"
}
```

Example response:

```
{
  "message": "Status updated",
  "status": "approved",
  "instructions": "",
  "deviceId": "team-blue-sim",
  "online": true,
  "speechLanguage": "de-DE"
}
```

Robot/PWA assignments must be read through their separate document endpoint, not from optional assignment metadata in telemetry acknowledgments.

Telemetry fields are merged into existing `statusData`, except:

- `lastSeenLocation` updates the top-level field and must be a non-empty string.
- `destinations`, if supplied, replaces the top-level list.
- `event` also updates recent event history. `"clear"` clears that history.

A telemetry POST refreshes `lastSeen` and sets `online` to true. For an HTTP-only simulator, sending it every 5-10 seconds keeps it within the current 20-second freshness threshold. MAC is checked when supplied; unregister always requires the matching MAC. Do not treat MAC values as credentials.

Destination upload body

```
[
  { "name": "Reception", "type": "usher", "x": 1.0, "y": 2.0, "z": 0.0 },
  { "name": "Waiting Room", "type": "table", "x": 4.0, "y": 5.0 }
]
```

Send non-empty `name` and `type` strings; coordinates are optional numbers. The type is free-form (for example `usher`, `table`, `transit`, `charge`). Upload **replaces the entire stored list**, not just matching entries. An empty array clears it. This stores gateway metadata; it does not create physical navigation points on a Pudu robot's onboard map.

8. Shared speech settings

Method	Route	Body	Success / behavior
GET	`/api/acu/settings`	None	`200`: `{ "speechLanguage": "de-DE" }` by default
PUT	`/api/acu/settings`	`{ "speechLanguage": "en-US" }`	`200`: saved settings; invalid language tag: `400`

The accepted format is 2-3 lowercase letters, a hyphen, then 2 uppercase letters, for example `de-DE`, `en-US`, or `hi-IN`. This affects all robots and is included in telemetry acknowledgment responses. It does not translate your workflow text or guarantee the device has that speech voice installed.

9. Copy-and-run integration examples

Read-only PowerShell dashboard query

```
$baseUrl = "http://192.168.1.100:3000"
$devices = Invoke-RestMethod -Uri "$baseUrl/api/acu/devices"
$devices | Select-Object deviceId, name, status, online, lastSeenLocation
```

Browser JavaScript: read devices, create and publish a workflow

Run against your team's approved device only. The create/publish calls below change server state; rerunning creation with the same ID produces `409`. No npm package is required.

```
const baseUrl = "http://192.168.1.100:3000"; // Replace with the event gateway.
const deviceId = "robot-001"; // Replace with your team's approved device.

async function api(path, { method = "GET", body } = {}) {
  const response = await fetch(`${baseUrl}${path}`, {
    method,
    headers: body === undefined ? {} : { "Content-Type": "application/json" },
    ...(body === undefined ? {} : { body: JSON.stringify(body) }),
  });
```

```
  const text = await response.text();
  if (!response.ok) {
    throw new Error(`${method} ${path}: HTTP ${response.status} ${text}`);
  }
  return text ? JSON.parse(text) : undefined;
}

const devices = await api("/api/acu/devices");
console.table(devices.map(({ deviceId, name, status, online }) =>
  ({ deviceId, name, status, online })));

const workflow = {
  WorkflowID: "team-blue-welcome",
  WorkflowName: "Team Blue Welcome",
  RobotId: deviceId,
  Steps: [{
    StepId: "Step-1",
    ScreenId: "Confirmation",
    Title: "Welcome to our demo",
    TTS: "Welcome to our demo",
    Buttons: {
      Yes: { Label: "Continue", Action: "Next" },
      No: { Label: "Back", Action: "Previous" },
    },
  }],
};

const created = await api("/api/workflows/robot", {
  method: "POST", body: workflow,
});
await api(
  `/api/acu/robot-workflows/${encodeURIComponent(created.WorkflowID)}` +
  `/assignments/${encodeURIComponent(deviceId)}`,
  { method: "POST" },
);
console.log("Published; check the robot client and telemetry for execution.");
```

For an `ACU.FE` development checkout, set `VITE_API_BASE_URL` to the organizer's gateway base URL (without `/api`) in its local environment configuration and restart the Vite development server. A separately hosted frontend needs a reachable gateway URL or an explicitly configured same-origin proxy.

10. Errors and troubleshooting

Most device errors return `{ "error": "..." }`. Robot workflow/media errors return Problem Details such as:

```
{
  "title": "Robot workflow operation failed",
  "status": 403,
```

```
  "detail": "Robot must be approved."
}
```

Framework validation errors can instead contain `title`, `status`, and an `errors` dictionary. Check HTTP status first, then `detail`, `error`, or `errors`. Do not automatically retry writes or treat every successful response as JSON.

Symptom / status	Check / action
Connection refused / timeout	Correct host and port, gateway process, Wi-Fi isolation/VPN routing, firewall, published Docker port
Works in Postman, fails in browser	Mixed content, CORS/proxy policy, certificate trust; inspect the browser network console
`400 Bad Request`	Body shape, required fields, JSON content type, destination names, media kind/extension
`403 Forbidden`	Device approval or supplied MAC mismatch; deployment proxy may add its own access controls
`404 Not Found`	Correct route prefix and identifier; missing assignment can also return this
`409 Conflict`	Duplicate registration/workflow ID or completing a workflow no longer assigned
`413 Payload Too Large`	Media/request limits at gateway or proxy
`503 Service Unavailable` on commands	Device is offline; check its telemetry/connectivity
Workflow created but robot shows nothing	Explicit publishing, correct robot, robot polling; republish after gateway restart
Robot is approved but offline	Approval is not a heartbeat; robot/simulator must send status updates
Destination list is empty	No map uploaded, cleared list, or unknown ID; confirm the device in `/api/acu/devices`
`500` or unexpected response	Retain status/body and report to organizers; do not loop retries against a shared robot

Some older repository endpoint tables omit the `/acu` or `/bot` prefix or describe obsolete assignment URLs. Use the exact routes in this guide and the running Swagger UI rather than guessing aliases.

Source references

- Gateway routing, JSON, CORS, and Swagger configuration
- Operator device APIs
- Robot registration and telemetry APIs
- Workflow and media APIs
- Robot workflow validation and assignment lifecycle

- Media limits and formats
- Destination APIs
- Shared settings APIs
- Robot workflow model
- Operator PWA API clients